

Government Welfare Scheme Management System

Project Report

UCS310

Database Management Systems

Submitted by:

Mehul Sharma 1024030932

Pramit Tiwari 1024030933

Nihchal Kaur 1024030938

B.Tech (2nd Year)

Submitted to:

Deepak Sharma



Department of Computer Science & Engineering
Thapar Institute Of Engineering & Technology

Introduction

Application Domain

This project falls under Public Administration & Social Welfare, focusing on digital governance (e-Governance) of social welfare schemes. The objective is to ensure that government resources reach eligible beneficiaries accurately, efficiently, and transparently through a centralized database system.

Motivation

The primary motivation is to eliminate “leakage” — the loss of funds due to corruption, duplication, or administrative inefficiency. Existing manual or fragmented digital systems suffer from:

- Identification Failure: Difficulty in verifying eligibility criteria.
- Slow Disbursement: Paper-based approvals delay fund transfers.
- Duplicate Benefits: Absence of centralized data allows multiple claims across departments.

Importance of DBMS (Over File-Based Systems)

Manual files and spreadsheets are inadequate for large-scale, multi-user systems. A DBMS provides:

- Reduction of Data Redundancy through Normalization: Citizen data is stored once and linked using foreign keys, ensuring consistency.
- Concurrency Control: Multiple officials can access and update records simultaneously without data corruption.
- Data Integrity & Constraints: PRIMARY KEY, FOREIGN KEY, CHECK, and UNIQUE constraints enforce valid data entry.
- Advanced Querying & Reporting: SQL enables efficient generation of eligibility and distribution reports.
- Security & Privacy: Role-Based Access Control (RBAC) restricts access to sensitive information.

Additionally, aggregate analysis supports data-driven decision-making and improved allocation of public funds.

Problem Statement

Traditional welfare distribution systems suffer from inefficiency, lack of transparency, and data fragmentation. Separate departmental records cause duplication, inconsistencies, and difficulty in tracking beneficiary eligibility and fund disbursement. A centralized DBMS is required to ensure structured data management and transparent operations.

Limitations of the Existing System

- **Data Fragmentation:** Independent departmental records create inconsistencies.
- **Lack of Transparency:** Manual tracking makes misuse detection difficult.
- **Ineligible Beneficiaries:** Manual verification is error-prone.
- **Operational Inefficiency:** File-based systems are slow to search and maintain.

Scope of Database Usage

- **Centralization:** Unique citizen records using primary and foreign keys.
- **Automated Validation:** Constraints and triggers enforce eligibility rules.
- **Transaction Integrity:** ACID properties ensure consistent fund updates.
- **Audit & Reporting:** SQL queries enable monitoring and transparency.

Objectives of the Project

The primary goal is to design a secure and structured backend database system for welfare management. The objectives are:

- Design a comprehensive ER model identifying entities such as Citizens, Departments, Schemes, and Applications.
- Convert the ER model into relational tables with appropriate data types.
- Apply Normalization (up to 3NF) to remove redundancy and anomalies.
- Implement data integrity using constraints (NOT NULL, UNIQUE, CHECK, FOREIGN KEY).
- Develop PL/SQL triggers and stored procedures for eligibility validation and fund disbursement.
- Ensure transactional consistency using COMMIT and ROLLBACK operations.

Scope of the Project

This project focuses on the backend database architecture for a welfare management system.

Functional Scope

- Citizen registration and profile management
- Scheme creation and eligibility definition
- Application tracking (Pending/Approved/Rejected)
- Fund disbursement recording and budget updates

User Roles

- Administrator: Full DDL/DML control.
- Department Officer: Manage schemes and application status.
- Citizen: View schemes and track application status.

Modules

- Identity Module: Citizen data management.
- Eligibility Module: Rule validation using PL/SQL.
- Disbursement Module: Budget-controlled payment processing.
- Audit Module: Trigger-based logging for transparency.

Database Design (Updated)

Entity–Relationship (ER) Model

The ER model of the Govt Welfare Scheme Management System has been updated to include additional entities for better transparency, document verification, and audit tracking. Along with the core welfare distribution process, the system now supports document management and activity logging.

The main entities are:

CITIZEN

- Citizen_ID (PK)
- Name
- Gender
- Category
- DOB
- Income
- Aadhaar_No (Unique)
- Region
- Occupation

Updated:

Added **Occupation** field for better citizen profiling and eligibility analysis.

SYSTEM_USER

- Login_ID (PK)
- Citizen_ID (FK, Optional & Unique)
- Username
- Password
- Email
- Phone_No
- Role

Updated:

Entity name changed from USER → SYSTEM_USER to avoid Oracle reserved keyword conflict.

DEPARTMENT

- Dept_ID (PK)
- Dept_Name
- HOD_Name
- Budget_Allocated

Updated:

Added UNIQUE constraint later on Dept_Name using ALTER TABLE.

SCHEME

- Scheme_ID (PK)
- Dept_ID (FK)
- Title
- Min_Age
- Max_Age
- Max_Income
- Benefit_Amount

No major structural changes.

APPLICATION

- App_ID (PK)
- Citizen_ID (FK)
- Scheme_ID (FK)
- Status (Pending / Approved / Rejected)
- Apply_Date

Core junction entity connecting Citizens and Schemes.

PAYMENT

- Txn_ID (PK)
- App_ID (FK, UNIQUE)
- Amount
- Pay_Date
- Payment_Mode

Updated:

Column name changed from **Mode** → **Payment_Mode** to avoid Oracle reserved keyword issues.

Also, **App_ID** is **UNIQUE** to ensure one approved application receives only one payment.

DOCUMENT

- Doc_ID (PK)
- Citizen_ID (FK)
- Document_Type
- Verification_Status
- Upload_Date

Newly Added:

This entity stores citizen verification documents such as:

- Income Certificate
- Aadhaar Verification
- Caste Certificate
- Residence Proof

This improves eligibility validation and transparency.

AUDIT_LOG

- Log_ID (PK)
- Action_Performed
- Action_Date
- Performed_By
- Remarks

Newly Added:

This entity records system activities such as:

- application submission
- approval actions
- payment release
- status updates

This improves monitoring, accountability, and transparency in government fund distribution.

Relationships

One-to-Many (1:M)

One Department manages multiple Schemes

A single department can launch and manage several welfare schemes.

Many-to-Many (M:N)

Citizens and Schemes

A citizen can apply for multiple schemes, and one scheme can have multiple applicants.

This relationship is resolved through the **APPLICATION** entity

One-to-One (1:0..1)

Application and Payment

Each approved application generates at most one payment record.

This is enforced using UNIQUE constraint on App_ID in Payment table.

Optional One-to-One (1:1)

Citizen and System_User

A citizen may optionally have one login account in the system.

One-to-Many (1:M)

Citizen and Document

One citizen can upload multiple verification documents.

This improves validation and record management.

One-to-Many (1:M)

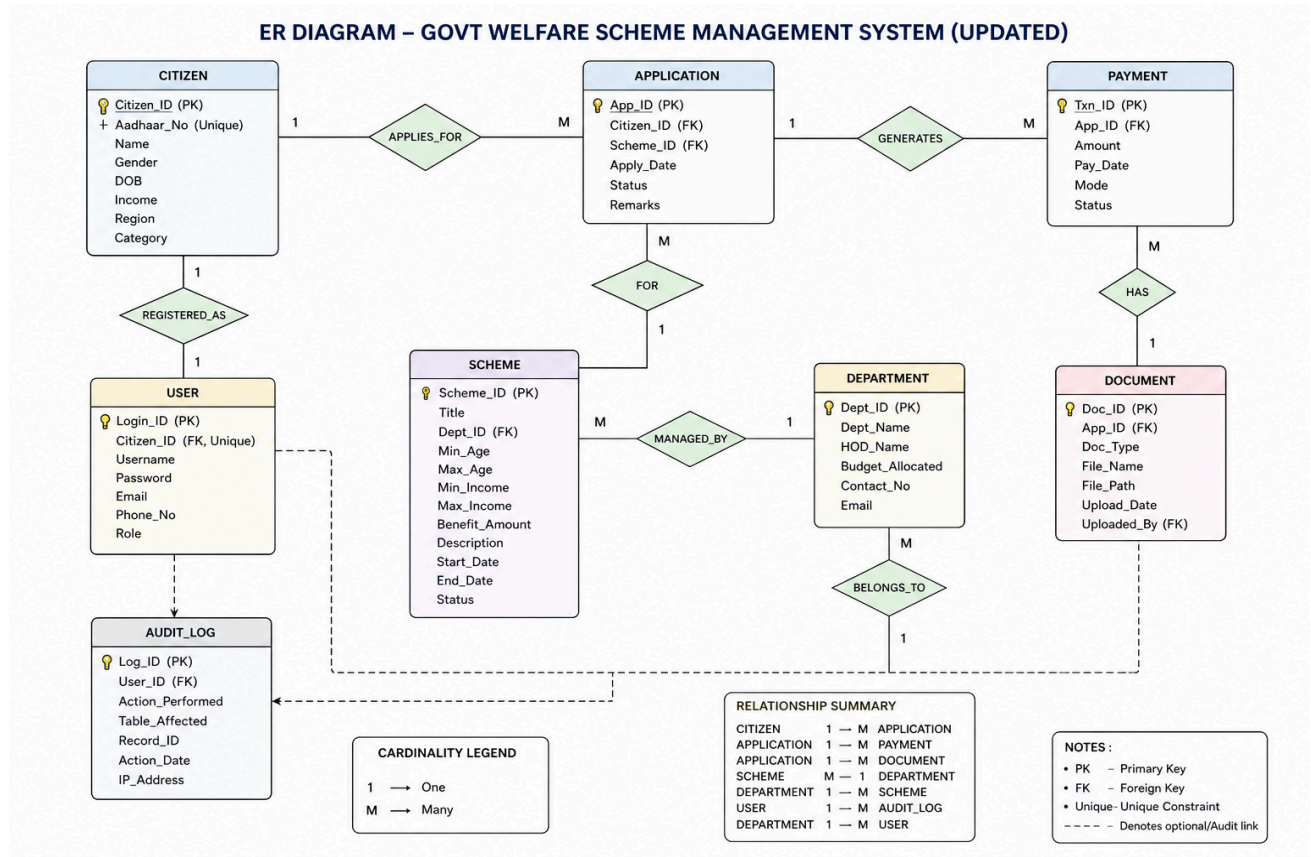
System Activity and Audit_Log

Multiple system events are recorded in Audit_Log for monitoring and administrative control.

Relational Schema (Updated)

- Citizens
(Citizen_ID, Name, Gender, Category, DOB, Income, Aadhaar_No, Region, Occupation)
- System_Users
(Login_ID, Citizen_ID, Username, Password, Email, Phone_No, Role)
- Departments
(Dept_ID, Dept_Name, HOD_Name, Budget_Allocated)
- Schemes
(Scheme_ID, Dept_ID, Title, Min_Age, Max_Age, Max_Income, Benefit_Amount)
- Applications
(App_ID, Citizen_ID, Scheme_ID, Status, Apply_Date)
- Payments
(Txn_ID, App_ID, Amount, Pay_Date, Payment_Mode)
- Documents
(Doc_ID, Citizen_ID, Document_Type, Verification_Status, Upload_Date)
- Audit_Log
(Log_ID, Action_Performed, Action_Date, Performed_By, Remarks)

ER DIAGRAM



Normalization

Normalization is applied up to **Third Normal Form (3NF)** to eliminate redundancy and ensure data integrity in the Govt Welfare Scheme Management System. Since the project now includes document verification and audit tracking, normalization also helps maintain consistency across citizen records, applications, payments, and system logs.

First Normal Form (1NF)

Goal:

Ensure atomicity of attributes (no repeating groups or multi-valued fields).

Each column stores a single value, and each record is uniquely identifiable.

For example, a citizen applying to multiple schemes is stored as separate rows in the **Applications** table instead of combining multiple Scheme_IDs in one field.

Similarly, if a citizen uploads multiple verification documents, each document is stored as a separate row in the **Document** table rather than storing multiple document names in one attribute.

This ensures proper structure and removes repeating groups.

Second Normal Form (2NF)

Goal:

Eliminate partial dependencies (applicable when composite keys exist).

In the **Applications** table, the relationship between Citizen and Scheme is conceptually based on:

(Citizen_ID + Scheme_ID)

Attributes such as:

- Scheme_Name
- Department_ID
- Benefit_Amount

depend only on:

Scheme_ID

not on the full relationship.

These attributes are moved to the **Schemes** table.

The **Applications** table retains only application-specific details such as:

- App_ID
- Application_Date
- Status

This reduces redundancy and improves maintainability.

Third Normal Form (3NF)

Goal:

Eliminate transitive dependencies (non-key attributes should not depend on other non-key attributes).

For example:

$\text{Scheme_ID} \rightarrow \text{Dept_ID} \rightarrow \text{Dept_Name}$

Here, **Dept_Name** depends on **Dept_ID**, not directly on **Scheme_ID**.

Therefore, **Dept_Name** is stored only in the **Departments** table, and the **Schemes** table contains only **Dept_ID** as a foreign key.

Similarly, audit details are stored separately in the **Audit_Log** table instead of repeating action information across multiple operational tables.

This ensures minimal redundancy, prevents update anomalies, and maintains consistency throughout the database.

Database Implementation

SQL Implementation

- DDL commands (CREATE, ALTER, DROP)

```
CREATE TABLE Citizen (  
    Citizen_ID NUMBER PRIMARY KEY,  
    Name VARCHAR2(50) NOT NULL,  
    Gender VARCHAR2(10) CHECK (Gender IN ('Male', 'Female', 'Other')),  
    Category VARCHAR2(30),  
    DOB DATE,  
    Income NUMBER CHECK (Income >= 0),  
    Aadhaar_No VARCHAR2(20) UNIQUE NOT NULL,  
    Region VARCHAR2(50)  
);  
  
CREATE TABLE System_User (  
    Login_ID NUMBER PRIMARY KEY,  
    Citizen_ID NUMBER UNIQUE,  
    Username VARCHAR2(50) UNIQUE NOT NULL,  
    Password VARCHAR2(50) NOT NULL,  
    Email VARCHAR2(100) UNIQUE,  
    Phone_No VARCHAR2(15),  
    Role VARCHAR2(20) CHECK (Role IN ('Admin', 'Officer', 'Citizen')),  
  
    CONSTRAINT fk_user_citizen  
    FOREIGN KEY (Citizen_ID)  
    REFERENCES Citizen(Citizen_ID)  
);  
  
CREATE TABLE Department (  
    Dept_ID NUMBER PRIMARY KEY,  
    Dept_Name VARCHAR2(100) NOT NULL,  
    HOD_Name VARCHAR2(50),  
    Budget_Allocated NUMBER CHECK (Budget_Allocated >= 0)  
);
```

```

ALTER TABLE Citizen
ADD Occupation VARCHAR2(50);

ALTER TABLE Audit_Log
MODIFY Remarks VARCHAR2(500);

ALTER TABLE Department
ADD CONSTRAINT unique_dept_name
UNIQUE (Dept_Name);

```

- DML(INSERT, UPDATE, DELETE for managing records)

```

INSERT INTO Department VALUES
(10, 'Social Welfare', 'R.K. Mehta', 5000000);

INSERT INTO Department VALUES
(20, 'Education', 'Pooja Sharma', 3000000);

INSERT INTO Department VALUES
(30, 'Health Department', 'Dr. S. Arora', 4000000);

INSERT INTO Department VALUES
(40, 'Women Welfare', 'Anita Kapoor', 3500000);

INSERT INTO Scheme VALUES
(201, 10, 'Old Age Pension', 60, 100, 200000, 5000);

INSERT INTO Scheme VALUES
(202, 20, 'Scholarship Scheme', 18, 30, 300000, 10000);

INSERT INTO Scheme VALUES
(203, 30, 'Health Support Scheme', 18, 65, 250000, 15000);

INSERT INTO Scheme VALUES
(204, 40, 'Widow Pension Scheme', 25, 60, 200000, 7000);

INSERT INTO Scheme VALUES
(205, 10, 'Farmer Support Scheme', 21, 60, 180000, 12000);

INSERT INTO Application VALUES
(301, 1, 202, 'Pending',
TO_DATE('2026-04-10', 'YYYY-MM-DD'));

INSERT INTO Application VALUES
(302, 2, 202, 'Approved',
TO_DATE('2026-04-12', 'YYYY-MM-DD'));

```

```
UPDATE Citizen
SET Income = 275000
WHERE Citizen_ID = 1;

UPDATE Application
SET Status = 'Approved'
WHERE App_ID = 301;

UPDATE Department
SET Budget_Allocated = 5500000
WHERE Dept_ID = 10;

DELETE FROM Document
WHERE Doc_ID = 502;

DELETE FROM Audit_Log
WHERE Log_ID = 601;

COMMIT;
```

Queries

Query 1: Citizens and Their Applied Schemes (JOIN)

Purpose:

Display citizen details along with the welfare schemes they have applied for.

```
--Query 1: Citizens and Their Applied Schemes (JOIN)

SELECT
    c.Citizen_ID,
    c.Name,
    c.Category,
    s.Title AS Scheme_Name,
    a.Status
FROM Citizen c
JOIN Application a
    ON c.Citizen_ID = a.Citizen_ID
JOIN Scheme s
    ON a.Scheme_ID = s.Scheme_ID;
```

Query Result 1 x

SQL | All Rows Fetched: 7 in 0.006 seconds

	CITIZEN_ID	NAME	CATEGORY	SCHEME_NAME	STATUS
1	1	Rahul Sharma	General	Scholarship Scheme	Approved
2	2	Priya Verma	OBC	Scholarship Scheme	Approved
3	3	Amanpreet Singh	SC	Old Age Pension	Pending
4	4	Simran Kaur	General	Health Support Scheme	Approved
5	5	Harpreet Singh	OBC	Farmer Support Scheme	Pending
6	6	Neha Gupta	General	Scholarship Scheme	Approved
7	2	Priya Verma	OBC	Widow Pension Scheme	Approved

Query 2: Approved Applications with Payment Details (Multiple JOIN)

Purpose:

Display approved applications and corresponding payment details.

<pre>-- Query 2: Approved Applications with Payment Details (Multiple JOIN) SELECT a.App_ID, c.Name, s.Title AS Scheme_Name, p.Amount, p.Pay_Date, p.Payment_Mode FROM Application a JOIN Citizen c ON a.Citizen_ID = c.Citizen_ID JOIN Scheme s ON a.Scheme_ID = s.Scheme_ID JOIN Payment p ON a.App_ID = p.App_ID WHERE a.Status = 'Approved';</pre>																																									
Query Result x SQL All Rows Fetched: 4 in 0.005 seconds <table> <tr> <th></th><th>APP_ID</th><th>NAME</th><th>SCHEME_NAME</th><th>AMOUNT</th><th>PAY_DATE</th><th>PAYMENT_MODE</th></tr> <tr> <td>1</td><td>302</td><td>Priya Verma</td><td>Scholarship Scheme</td><td>10000</td><td>20-04-26</td><td>Bank Transfer</td></tr> <tr> <td>2</td><td>304</td><td>Simran Kaur</td><td>Health Support Scheme</td><td>15000</td><td>25-04-26</td><td>Bank Transfer</td></tr> <tr> <td>3</td><td>306</td><td>Neha Gupta</td><td>Scholarship Scheme</td><td>10000</td><td>26-04-26</td><td>UPI</td></tr> <tr> <td>4</td><td>307</td><td>Priya Verma</td><td>Widow Pension Scheme</td><td>7000</td><td>27-04-26</td><td>Bank Transfer</td></tr> </table>								APP_ID	NAME	SCHEME_NAME	AMOUNT	PAY_DATE	PAYMENT_MODE	1	302	Priya Verma	Scholarship Scheme	10000	20-04-26	Bank Transfer	2	304	Simran Kaur	Health Support Scheme	15000	25-04-26	Bank Transfer	3	306	Neha Gupta	Scholarship Scheme	10000	26-04-26	UPI	4	307	Priya Verma	Widow Pension Scheme	7000	27-04-26	Bank Transfer
	APP_ID	NAME	SCHEME_NAME	AMOUNT	PAY_DATE	PAYMENT_MODE																																			
1	302	Priya Verma	Scholarship Scheme	10000	20-04-26	Bank Transfer																																			
2	304	Simran Kaur	Health Support Scheme	15000	25-04-26	Bank Transfer																																			
3	306	Neha Gupta	Scholarship Scheme	10000	26-04-26	UPI																																			
4	307	Priya Verma	Widow Pension Scheme	7000	27-04-26	Bank Transfer																																			

Query 3: Total Applications per Scheme (GROUP BY)

Purpose:

Count the total number of applications received for each welfare scheme.

```
--Query 3: Total Applications per Scheme (GROUP BY)
```

```

SELECT
    s.Title AS Scheme_Name,
    COUNT(a.App_ID) AS Total_Applications
FROM Scheme s
LEFT JOIN Application a
    ON s.Scheme_ID = a.Scheme_ID
GROUP BY s.Title;

```

	APP_ID	NAME	SCHEME_NAME	AMOUNT	PAY_DATE	PAYMENT_MODE
1	302	Priya Verma	Scholarship Scheme	10000	20-04-26	Bank Transfer
2	304	Simran Kaur	Health Support Scheme	15000	25-04-26	Bank Transfer
3	306	Neha Gupta	Scholarship Scheme	10000	26-04-26	UPI
4	307	Priya Verma	Widow Pension Scheme	7000	27-04-26	Bank Transfer

Query 4: Schemes Having More Than One Application (HAVING)

Purpose:

Use HAVING clause to filter schemes with multiple applicants.

```
--Query 4: Schemes Having More Than One Application (HAVING)
```

```

SELECT
    s.Title AS Scheme_Name,
    COUNT(a.App_ID) AS Total_Applications
FROM Scheme s
JOIN Application a
    ON s.Scheme_ID = a.Scheme_ID
GROUP BY s.Title
HAVING COUNT(a.App_ID) > 1;

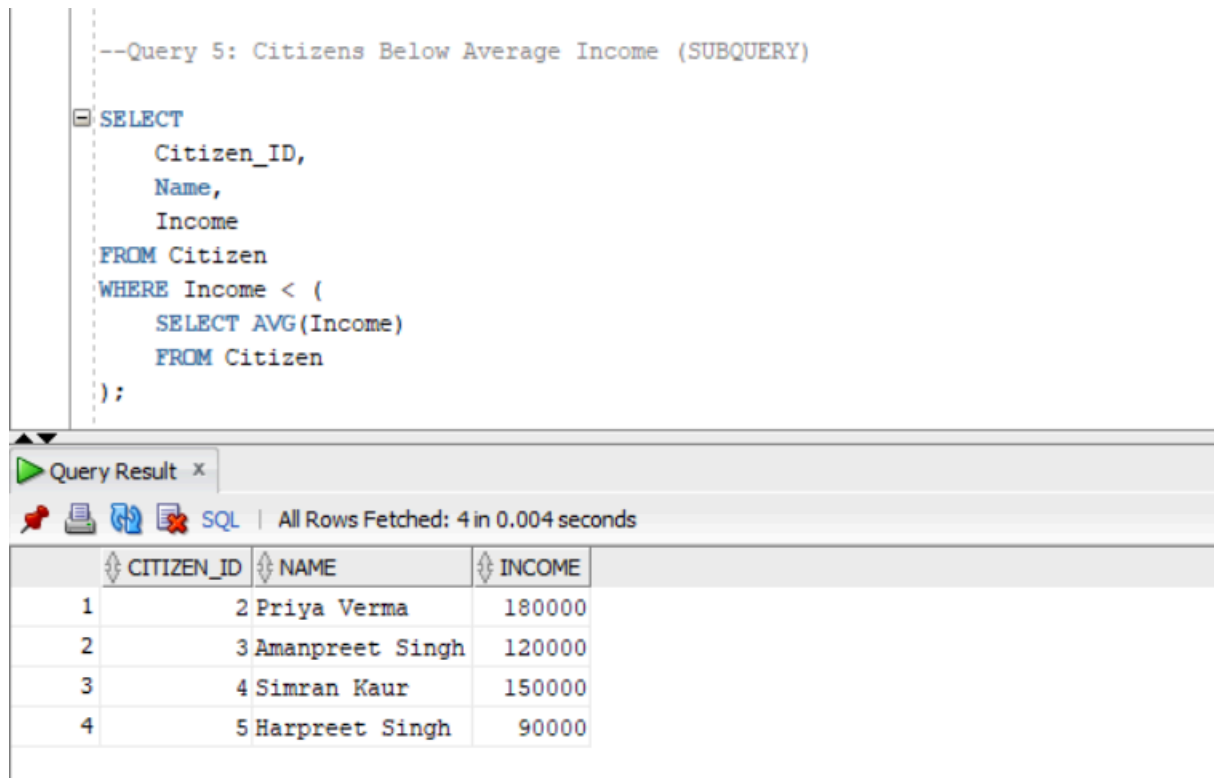
```

	SCHEME_NAME	TOTAL_APPLICATIONS
1	Scholarship Scheme	3

Query 5: Citizens Below Average Income (SUBQUERY)

Purpose:

Find citizens whose income is below the average income of all citizens.



```
--Query 5: Citizens Below Average Income (SUBQUERY)

SELECT
    Citizen_ID,
    Name,
    Income
FROM Citizen
WHERE Income < (
    SELECT AVG(Income)
    FROM Citizen
);
```

Query Result x

SQL | All Rows Fetched: 4 in 0.004 seconds

	CITIZEN_ID	NAME	INCOME
1	2	Priya Verma	180000
2	3	Amanpreet Singh	120000
3	4	Simran Kaur	150000
4	5	Harpreet Singh	90000

Query 6: Create a View for Approved Applications (VIEW)

Purpose:

Create a reusable view for approved applications.

--Query 6: Create a View for Approved Applications (VIEW)

```
CREATE VIEW Approved_Applications AS
SELECT
    a.App_ID,
    c.Name,
    s.Title AS Scheme_Name,
    a.Status
FROM Application a
JOIN Citizen c
    ON a.Citizen_ID = c.Citizen_ID
JOIN Scheme s
    ON a.Scheme_ID = s.Scheme_ID
WHERE a.Status = 'Approved';

SELECT * FROM Approved_Applications;
```

Query Result x

SQL | All Rows Fetched: 5 in 0.019 seconds

	APP_ID	NAME	SCHEME_NAME	STATUS
1	301	Rahul Sharma	Scholarship Scheme	Approved
2	302	Priya Verma	Scholarship Scheme	Approved
3	307	Priya Verma	Widow Pension Scheme	Approved
4	304	Simran Kaur	Health Support Scheme	Approved
5	306	Neha Gupta	Scholarship Scheme	Approved

PL/SQL Components

- Function: Checks citizen eligibility

```
CREATE OR REPLACE FUNCTION Check_Eligibility (  
    p_Citizen_ID IN NUMBER,  
    p_Scheme_ID IN NUMBER  
)  
RETURN VARCHAR2  
IS  
    v_income Citizen.Income%TYPE;  
    v_max_income Scheme.Max_Income%TYPE;  
BEGIN  
    -- Fetch citizen income  
    SELECT Income  
    INTO v_income  
    FROM Citizen  
    WHERE Citizen_ID = p_Citizen_ID;  
  
    -- Fetch scheme max income limit  
    SELECT Max_Income  
    INTO v_max_income  
    FROM Scheme  
    WHERE Scheme_ID = p_Scheme_ID;  
  
    -- Eligibility check  
    IF v_income <= v_max_income THEN  
        RETURN 'Eligible';  
    ELSE  
        RETURN 'Not Eligible';  
    END IF;  
  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        RETURN 'Invalid Citizen or Scheme ID';  
  
    WHEN OTHERS THEN  
        RETURN 'Error Occurred';  
END;  
/  
  
SELECT Check_Eligibility(1, 202)  
FROM dual;
```

Query Result x

SQL | All Rows Fetched: 1 in 0.012 seconds

CHECK_ELIGIBILITY(1,202)

1 Eligible

- Trigger: Validates eligibility before application insertion

```
CREATE OR REPLACE TRIGGER trg_check_eligibility
BEFORE INSERT ON Application
FOR EACH ROW
DECLARE
    v_result VARCHAR2(30);
BEGIN
    -- Call function to check eligibility
    v_result := Check_Eligibility(:NEW.Citizen_ID, :NEW.Scheme_ID);

    -- If not eligible, stop insertion
    IF v_result <> 'Eligible' THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Citizen is not eligible for this scheme.'
        );
    END IF;
END;
/
```

```
INSERT INTO Application VALUES
(308, 6, 205, 'Pending',
TO_DATE('2026-05-01','YYYY-MM-DD'));
```

Script Output x

Task completed in 0.155 seconds

Error starting at line : 413 in command -

```
INSERT INTO Application VALUES
(308, 6, 205, 'Pending',
TO_DATE('2026-05-01','YYYY-MM-DD'))
```

Error at Command Line : 413 Column : 13

Error report -

SQL Error: ORA-20001: Citizen is not eligible for this scheme.

ORA-06512: at "SYSTEM.TRG_CHECK_ELIGIBILITY", line 9

ORA-04088: error during execution of trigger 'SYSTEM.TRG_CHECK_ELIGIBILITY'

- Stored Procedure: Handles approval and fund disbursement

```

CREATE OR REPLACE PROCEDURE Approve_Application (
    p_App_ID IN NUMBER,
    p_Amount IN NUMBER,
    p_Mode IN VARCHAR2
)
IS
BEGIN
    -- Step 1: Update application status
    UPDATE Application
    SET Status = 'Approved'
    WHERE App_ID = p_App_ID;

    -- Step 2: Insert payment record
    INSERT INTO Payment (
        Txn_ID,
        App_ID,
        Amount,
        Pay_Date,
        Payment_Mode
    )
    VALUES (
        Payment_seq.NEXTVAL,
        p_App_ID,
        p_Amount,
        SYSDATE,
        p_Mode
    );

    COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

```

```

BEGIN
    Approve_Application(
        303,
        5000,
        'UPI'
    );
END;
/

```

Script Output x

Task completed in 0.116 seconds

PL/SQL procedure successfully completed.

- Cursor: Processes bulk disbursement of approved applications

```
DECLARE
    -- Variables to store fetched values
    v_App_ID Application.App_ID%TYPE;
    v_Citizen_ID Application.Citizen_ID%TYPE;
    v_Scheme_ID Application.Scheme_ID%TYPE;
    v_Status Application.Status%TYPE;

    -- Cursor declaration
    CURSOR pending_apps IS
        SELECT App_ID, Citizen_ID, Scheme_ID, Status
        FROM Application
        WHERE Status = 'Pending';

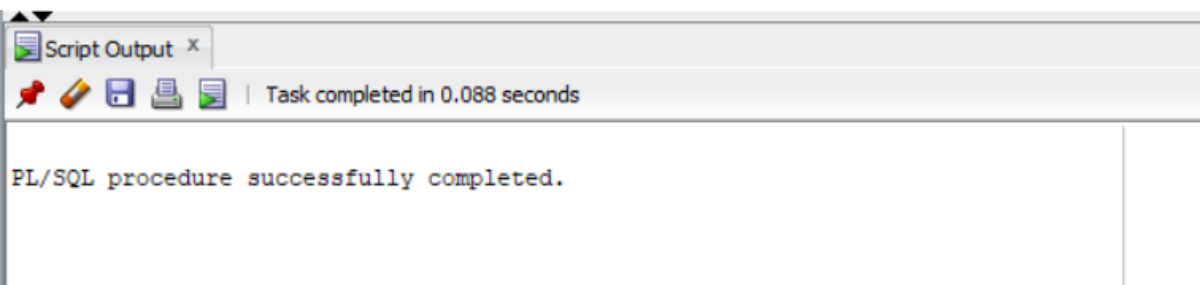
BEGIN
    OPEN pending_apps;

    LOOP
        FETCH pending_apps
        INTO v_App_ID, v_Citizen_ID, v_Scheme_ID, v_Status;

        EXIT WHEN pending_apps%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'App ID: ' || v_App_ID ||
            ' | Citizen ID: ' || v_Citizen_ID ||
            ' | Scheme ID: ' || v_Scheme_ID ||
            ' | Status: ' || v_Status
        );
    END LOOP;

    CLOSE pending_apps;
END;
/
```



- Exception Handling

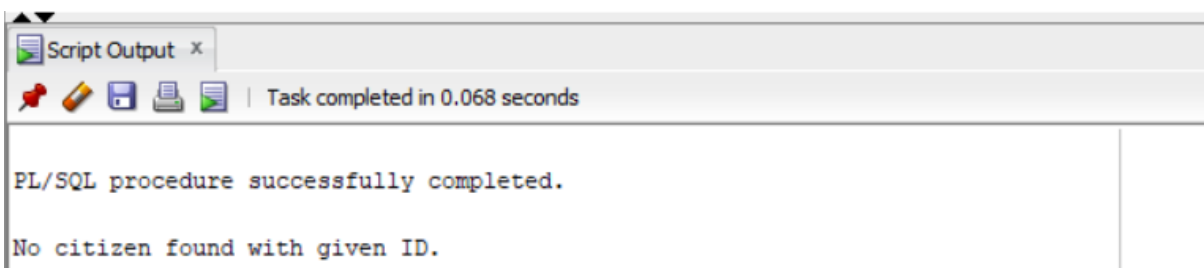
```
DECLARE
    v_name Citizen.Name%TYPE;
BEGIN
    -- Try to fetch citizen name
    SELECT Name
    INTO v_name
    FROM Citizen
    WHERE Citizen_ID = 999;

    DBMS_OUTPUT.PUT_LINE(
        'Citizen Name: ' || v_name
    );

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE(
            'No citizen found with given ID.'
        );

    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Multiple records found.'
        );

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );
END;
/
```



Transaction Management & Concurrency

To ensure reliability and consistency, the system implements:

ACID Properties:

- Atomicity: COMMIT and ROLLBACK ensure that either all updates succeed or none are applied.
- Consistency: Budget constraints ensure that total disbursement does not exceed scheme allocation.
- Isolation: Row-level locking prevents conflicts during simultaneous updates.
- Durability: Committed transactions are permanently stored.

Tools & Technologies Used

- DBMS: Oracle 19c / MySQL 8.0
- Languages: SQL and PL/SQL
- Design Tools: Oracle SQL Developer / MySQL Workbench
- Environment: SQL Command Line and GUI tools

Expected Outcomes

- A normalized database (up to 3NF) eliminating redundancy.
- Automated eligibility validation using PL/SQL triggers.
- Transaction-safe financial updates using ACID principles.
- Efficient reporting using advanced SQL queries.